

Funcionamiento del proyecto Papaws

Arquitectura de microservicios, servicios y flujo de datos

Equipo Papaws

June 17, 2026

Abstract

Papaws es un sistema de gestión para un refugio veterinario de animales rescatados. Está construido como un conjunto de **microservicios** en **ASP.NET Core** (.NET 10) con persistencia en **Apache Cassandra**, y un **frontend** en **React + TypeScript + Vite**. Este documento explica qué hace cada servicio, cómo están conectados entre sí, qué información muestra la interfaz y cómo se despliega todo el conjunto. Para el detalle del modelo de datos físico (tablas y claves de Cassandra) consúltase el documento complementario *Esquema de tablas en Cassandra*.

Contents

1	Visión general	2
1.1	Diagrama de componentes	2
2	Tecnologías utilizadas	3
3	Pawpaws-Animales	3
3.1	Responsabilidades	3
3.2	Estructura interna	3
3.3	Datos que administra	3
3.4	Endpoints principales	4
4	Pawpaws-Consulta	4
4.1	Responsabilidades	4
4.2	Estados de una consulta	4
4.3	Gestión de inventario	5
4.4	Endpoints principales	5
5	Pawpaws-Reportes	5
5.1	Cómo funciona	5
5.2	Catálogo de reportes	6
6	Pawpaws-Front (interfaz web)	6
6.1	Páginas y qué muestran	6
6.2	Organización del código	6
6.3	Enrutamiento y protección por rol	7
7	Autenticación y autorización	7
7.1	Flujo del token	7
7.2	Roles	7
8	Comunicación entre servicios	7

9 Persistencia (Cassandra)	8
10 Sembrado de datos de prueba	8
10.1 Datos aleatorios con Bogus	8
10.2 Datos fijos (demo curada)	8
11 Despliegue	9
12 Flujo de ejemplo: registrar una consulta completa	9

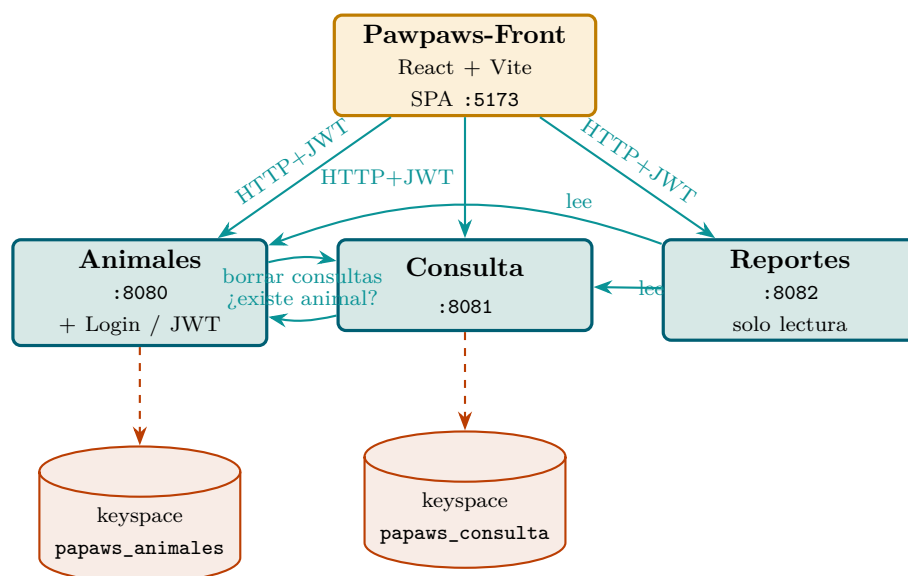
1 Visión general

El sistema se divide en cuatro piezas desplegables independientes, más dos bases de datos Cassandra:

- **Pawpaws-Animales** (puerto 8080): gestiona *rescatistas* y *animales*. Es además el **emisor de los tokens de autenticación** (login) para todo el sistema.
- **Pawpaws-Consulta** (puerto 8081): gestiona *consultas* veterinarias y sus catálogos asociados: *veterinarios*, *servicios* y *productos* (inventario).
- **Pawpaws-Reportes** (puerto 8082): servicio de *solo lectura* que agrega y combina datos de los otros dos servicios para producir reportes; no tiene base de datos propia.
- **Pawpaws-Front** (puerto 5173 en desarrollo): la aplicación web de una sola página (SPA) que consume las tres APIs.

Idea clave. Cada microservicio es dueño de sus propios datos y de su propio keyspace de Cassandra. Cuando un servicio necesita información de otro, *no* accede a su base de datos: hace una llamada HTTP a su API, reenviando el token del usuario. Reportes es un caso especial: *solo* habla por HTTP con los otros dos y no almacena nada.

1.1 Diagrama de componentes



Líneas continuas: llamadas HTTP (REST) con cabecera Authorization: Bearer <token>.
Líneas discontinuas: acceso del servicio a su keyspace de Cassandra.

2 Tecnologías utilizadas

Capa	Tecnología
Backend	ASP.NET Core (.NET 10), controladores Web API, Swagger/OpenAPI.
Persistencia	Apache Cassandra 4.1, driver <code>CassandraCSharpDriver</code> , <code>PreparedStatement</code> .
Autenticación	JWT (HMAC-SHA256) emitido por el servicio de Animales; validado por los tres servicios.
Frontend	React 18, TypeScript, Vite, React Router, Tailwind CSS, Recharts (gráficos), lucide-react (iconos).
Infraestructura	Docker / Docker Compose; dos nodos Cassandra; healthchecks.

3 Pawpaws-Animales

Es el servicio **central** y la puerta de entrada de la autenticación.

3.1 Responsabilidades

- Administrar **rescatistas** (personas u organizaciones que rescatan animales) y los **animales** a su cargo.
- Emitir el **token JWT** de login que usa todo el sistema (`POST /api/auth/login`).
- Mantener la coherencia al **eliminar** entidades: al borrar un animal ordena en cascada el borrado de sus consultas en el servicio de Consulta; al borrar un rescatista, reasigna sus animales a otro (por defecto al rescatista interno «Refugio»).

3.2 Estructura interna

El servicio sigue una arquitectura en capas clásica:

- **Controllers** — exponen los endpoints REST y aplican autorización por rol.
- **Services** — contienen la lógica de negocio y las consultas a Cassandra mediante `PreparedStatement`.
- **Models** — entidades de dominio (`Animal`, `Rescatista`).
- **DTOs** — objetos de entrada/salida de la API (separados del modelo).
- **Data/CassandraSchema** — crea el keyspace y las tablas al arrancar (idempotente, con migraciones `ALTER ... IF NOT EXISTS`).
- **Security** — opciones de JWT, usuarios semilla y roles.
- **Common** — manejador global de excepciones, healthcheck de Cassandra, mappers y paginación.

3.3 Datos que administra

- **Rescatista**: nombre, teléfono, correo, organización, zona de operación; banderas **activo** (borrado lógico) y **oculto** (el «Refugio» interno no se muestra en listados).
- **Animal**: nombre, especie, peso actual, fecha de ingreso y el rescatista asociado. Se almacena en dos tablas desnormalizadas (`animales_by_id` y `animales_by_rescatista`) para poder consultarlo por ambas claves.

3.4 Endpoints principales

Método	Ruta	Descripción
POST	/api/auth/login	Autentica y devuelve el token JWT (anónimo).
GET	/api/rescatistas	Lista rescatistas activos (paginado).
POST	/api/rescatistas	Crea un rescatista.
PUT	/api/rescatistas/{id}	Actualiza un rescatista.
DELETE	/api/rescatistas/{id}	Baja lógica; reasigna sus animales (?reassignarA=).
GET	/api/animales	Lista animales (paginado).
GET	/api/animales/{id}	Obtiene un animal (lo usa Consulta y Reportes).
GET	/api/animales/rescatista/{id}	Animales de un rescatista.
POST	/api/animales	Crea un animal.
PUT	/api/animales/{id}	Actualiza un animal.
DELETE	/api/animales/{id}	Borra el animal y, en cascada, sus consultas.
GET	/health	Healthcheck (estado de Cassandra).

4 Pawpaws-Consulta

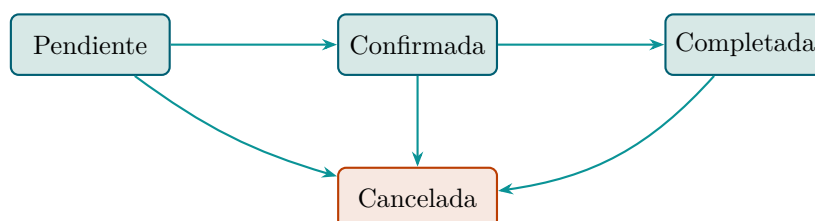
Gestiona el núcleo clínico del refugio: las consultas veterinarias y sus catálogos.

4.1 Responsabilidades

- Administrar **veterinarios**, **servicios** (procedimientos con precio y duración) y **productos** (inventario con **stock**).
- Gestionar el **ciclo de vida de una consulta**: creación, edición, reprogramación, cambio de estado, registro de diagnóstico y de productos usados, y cancelación.
- Validar contra el servicio de Animales que el **animal exista** antes de crear una consulta (llamada HTTP, no acceso directo a la otra BD).

4.2 Estados de una consulta

Una consulta transita por una máquina de estados controlada:



Reglas relevantes: una consulta **Cancelada** no cambia de estado; una **Completada** solo puede cancelarse. Registrar un diagnóstico o productos marca la consulta como **Completada**. Al **cancelar**, los productos que se habían registrado se **devuelven al stock** automáticamente.

4.3 Gestión de inventario

El registro de productos usados es transaccional en su validación: primero se valida *todo* (existencia, baja lógica y stock suficiente) y solo después se descuenta el stock, evitando descuentos a medias. Las cantidades se acumulan por producto para no perder registros previos.

4.4 Endpoints principales

Método	Ruta	Descripción
GET	/api/veterinarios	Catálogo de veterinarios.
GET	/api/servicios	Catálogo de servicios.
GET	/api/productos	Inventario de productos.
PUT	/api/productos/{id}/stock	Establece el stock de un producto.
GET	/api/consultas	Lista consultas (liviano).
GET	/api/consultas/{codigo}	Detalle de consulta (con servicios y productos).
GET	/api/consultas/animal/{id}	Consultas de un animal.
GET	/api/consultas/veterinario/{id}	Consultas de un veterinario.
POST	/api/consultas	Crea una consulta (valida animal en Animales).
PUT	/api/consultas/{codigo}/estado	Cambia el estado.
PUT	/api/consultas/{codigo}/reprogramar	Cambia la fecha/hora.
PUT	/api/consultas/{codigo}/diagnostico	Registra diagnóstico (completa).
POST	/api/consultas/{codigo}/productos	Registra productos usados (descuenta stock).
DELETE	/api/consultas/{codigo}	Cancela/elimina la consulta (restaura stock).
DELETE	/api/consultas/animal/{id}	Borrado en cascada (lo invoca Animales).

5 Pawpaws-Reportes

Servicio de **agregación de solo lectura**. No tiene base de datos: actúa como un *Backend-for-Frontend* de reportes que consulta por HTTP a Animales y a Consulta, combina los resultados y los devuelve ya formateados.

5.1 Cómo funciona

- Define dos clientes HTTP tipados: `AnimalesClient` (apunta a `:8080`) y `ConsultaClient` (apunta a `:8081`).
- `ReporteService` orquesta las llamadas: por ejemplo, para el reporte «consultas por animal» obtiene primero el animal de Animales y luego sus consultas de Consulta, y compone un único DTO de respuesta.
- Los DTOs de entrada externos viven en `DTOs/Externos` y los DTOs de salida del reporte (C1...C19) en `DTOs/Reportes`.

5.2 Catálogo de reportes

El servicio expone alrededor de 18 reportes numerados (C1–C19). Algunos ejemplos:

Cód.	Reporte
C1	Datos de un rescatista por id.
C2	Animales de un rescatista.
C3	Animales por especie.
C4	Consultas de un animal.
C5	Consultas de un veterinario.
C6	Detalle completo de una consulta.
C7	Consultas por estado.
C8	Servicios usados en una consulta.
C9	Productos usados en una consulta.
C10	Productos ordenados por stock.
C13	Veterinarios por especialidad.
C16	Consultas por fecha.
C17	Animales por nombre.
C19	Rescatistas por zona.

La interfaz consume estos reportes en `/api/reportes/...` y los muestra como tablas y gráficos.

6 Pawpaws-Front (interfaz web)

SPA en React que es lo único que ve el usuario. Se comunica con las tres APIs.

6.1 Páginas y qué muestran

Página	Contenido
Login	Formulario de acceso; obtiene el token de Animales.
Dashboard	Resumen general / inicio.
Rescatistas	Listado y CRUD de rescatistas (con reasignación al borrar).
Animales	Listado y CRUD de animales; filtrado por rescatista.
Veterinarios	Catálogo de veterinarios (módulo consultas).
Servicios	Catálogo de servicios.
Productos	Inventario y ajuste de stock.
Consultas	Listado de consultas y creación.
ConsultaDetalle	Detalle de una consulta: estado, diagnóstico, servicios y productos usados.
Reportes	Reportes C1–C19 visualizados con tablas y gráficos (Recharts).

6.2 Organización del código

- `api/client.ts` — capa HTTP base: añade el token, gestiona errores y el 401 (cierra sesión automáticamente).
- `api/endpoints.ts` — funciones tipadas por dominio (`rescatistasApi`, `consultasApi`, `reportesApi...`).
- `auth/` — `AuthContext` (estado de sesión) y *guards* de ruta (`ProtectedRoute`, `RoleRoute`).
- `pages/` — una página por módulo; `components/` — UI reutilizable; `hooks/` — `useFetch`; `types/` — tipos compartidos con el backend.

6.3 Enrutamiento y protección por rol

Las rutas están envueltas en `ProtectedRoute` (exige sesión) y el módulo de consultas, además, en `RoleRoute` (solo *Administrador* y *EncargadoConsultas*). El token se guarda en `localStorage`; si una API devuelve 401, la app emite un evento `papaws:unauthorized` y vuelve al login.

7 Autenticación y autorización

7.1 Flujo del token

1. El usuario hace login contra `POST :8080/api/auth/login`.
2. Animales valida las credenciales contra una lista de **usuarios semilla** (modelo simple con contraseña genérica de desarrollo) y firma un **JWT** con HMAC-SHA256.
3. El token incluye el `rol` del usuario y caduca (480 min por defecto).
4. El frontend lo almacena y lo envía como `Authorization: Bearer ...` en cada llamada.
5. Los **tres** servicios validan el mismo token con la **misma clave** (`Auth:Jwt:Key`); solo Animales lo *emite*.

7.2 Roles

Rol	Permisos
Administrador	Acceso completo a todos los módulos.
EncargadoRescatistas	Gestiona rescatistas y animales; lectura.
EncargadoConsultas	Gestiona el módulo clínico (consultas, veterinarios, servicios, productos).

La autorización se aplica con `[Authorize(Roles = ...)]` en los controladores del backend y con `RoleRoute` en el frontend.

8 Comunicación entre servicios

Los servicios nunca comparten base de datos; se coordinan por HTTP reenviando el token del usuario actual. Existen tres caminos:

1. **Consulta** → **Animales**: al crear una consulta, `AnimalReferenceService` llama a `GET /api/animales/{id}` para confirmar que el animal existe.
2. **Animales** → **Consulta**: al borrar un animal, `ConsultaReferenceService` llama a `DELETE /api/consultas/{id}` para eliminar en cascada sus consultas (y restaurar el stock asociado).
3. **Reportes** → **Animales y Consulta**: lecturas para componer los reportes.

Resiliencia. Las llamadas entre servicios usan *timeouts* cortos (5–10 s) y **reintentos con backoff** (hasta 3 intentos) ante fallos de red o errores 5xx transitorios, para no dejar peticiones colgadas ni romper las cascadas en silencio. Un 404 se trata como respuesta válida (p. ej. «el animal no existe»), no como error a reintentar.

9 Persistencia (Cassandra)

Cada servicio con estado tiene su propio **keyspace** (`papaws_animales` y `papaws_consulta`), ambos con `SimpleStrategy` y `replication_factor = 2` sobre un clúster de dos nodos. El esquema se crea automáticamente al arrancar cada servicio (`CassandraSchema.InitializeAsync`), de forma idempotente.

El modelado sigue el patrón *query-first* de Cassandra: los datos se desnormalizan en varias tablas (p.ej. `animales_by_id` y `animales_by_rescatista`, o las tablas-puntero `consulta_codigos_by_animal` / `..._by_veterinario`) para poder leer por distintas claves sin `ALLOW FILTERING`. El detalle completo de columnas y claves está en el documento *Esquema de tablas en Cassandra*.

10 Sembrado de datos de prueba

Para poblar el sistema con datos de prueba existen **dos mecanismos complementarios**, ambos protegidos por autenticación:

10.1 Datos aleatorios con Bogus

El servicio de Animales incluye `DatosPruebaService`, que usa la librería **Bogus** (`new Faker("es")`) para generar registros *aleatorios* y realistas en español: nombres, teléfonos, correos y ciudades falsos. Por cada rescatista crea entre 1 y 3 animales, con especie elegida al azar (*Puma, Loro, Tortuga, Oso, Mono*) y peso aleatorio.

Aspecto	Detalle
Endpoint	<code>POST /api/datos-prueba/animales/{cantidad}</code>
Rol	Solo Administrador .
Librería	Bogus 35.6.5 (paquete NuGet en <code>Pawpaws.Animales.csproj</code>).
Límites	cantidad entre 1 y 1000 rescatistas por solicitud.
Genera	Rescatistas + 1-3 animales por rescatista, todos con datos <i>aleatorios</i> .

Bogus solo se usa en Animales para la *carga masiva aleatoria*. Es ideal para pruebas de volumen, pero los datos no son reproducibles entre ejecuciones (cambian en cada llamada).

10.2 Datos fijos (demo curada)

De forma independiente, cada servicio expone `POST /api/seed` con un conjunto *fijo y determinista* pensado para una demo coherente:

- **Animales:** 8 rescatistas con nombre propio (María García, Carlos López...) y ~26 animales variados (perros, gatos, conejos, aves, exóticos). Devuelve los `animalIds` creados.
- **Consulta:** 6 veterinarios, 8 servicios, 10 productos y 25 consultas (`CONS-2024-001...`) en distintos estados, vinculadas a los `animalIds` que recibe del paso anterior.
- Ambos son **idempotentes**: si ya hay datos, no vuelven a sembrar.

El frontend orquesta este sembrado encadenado (`seedApi.animales()` y luego `seedApi.consulta(animalIds)`), de modo que las consultas queden asociadas a animales reales.

11 Despliegue

Todo el sistema se levanta con **Docker Compose** (`(docker-compose.yml)`):

- Dos contenedores Cassandra (`cassandra-animales` en `9042` y `cassandra-consulta` en `9142`) que forman un clúster, con *healthchecks*.
- Los servicios .NET arrancan **después** de que Cassandra esté sana (`depends_on: condition: service_healthy`). Consulta espera además a que Animales esté iniciado, y Reportes a ambos.
- La configuración (puntos de contacto de Cassandra, URLs de los otros servicios y la clave JWT compartida) se inyecta por **variables de entorno**.

En desarrollo, el frontend corre con Vite en `:5173` y apunta por defecto a `:8080`, `:8081` y `:8082` (configurable con las variables `VITE_API_*`). El CORS de los servicios autoriza explícitamente los orígenes `:5173` y `:4173`.

12 Flujo de ejemplo: registrar una consulta completa

1. El usuario inicia sesión → Animales emite el JWT.
2. Crea una consulta para un animal → Consulta valida con Animales que el animal existe y que el veterinario/servicios estén activos.
3. Atiende la consulta y registra los **productos usados** → Consulta valida stock, lo descuenta y marca la consulta como **Completada**.
4. Registra el **diagnóstico** e indicaciones de seguimiento.
5. Más tarde se consulta el reporte «*consultas por animal*» (*C4*) → Reportes combina datos de Animales y Consulta y los muestra en el frontend como tabla/gráfico.
6. Si se **cancela** la consulta, el stock de los productos registrados se **devuelve** automáticamente al inventario.

*Documento generado para describir el funcionamiento del proyecto Papaws.
Complementario a «Esquema de tablas en Cassandra».*